genai

Table of Contents

LLMs (Large Language Models)

What is an LLM?

A Large Language Model is a neural network trained on massive amounts of text data to understand and generate human-like language. It predicts the next token in a sequence.

LLM vs Applications

An LLM is the model itself. Applications like ChatGPT, Claude, or Copilot are interfaces that use LLMs.

Term	What it is	Example
LLM	The AI model	GPT-4, Claude 3.5, Llama
Application	The product that uses the model	ChatGPT, Claude, GitHub Copilot

Think of it as: the LLM is the engine, the application is the car.

How It Works

Tokens

Text is split into tokens (subwords, characters, or words) before processing.

```text
"Hello world" → ["Hello", " world"]
"unhappiness" → ["un", "happiness"]
```

- English: ~1 token ≈ 4 characters or 0.75 words
- Spanish: slightly more tokens per word due to diacritics

Context Window

The maximum number of tokens a model can process at once (input + output).

Model	Context Window
GPT-4o	128K tokens
Claude 3.5 Sonnet	200K tokens
Gemini 1.5 Pro	1M tokens

Context window = what the model can "see" at once. Beyond it, the model loses information.

Transformer Architecture

The architecture behind most modern LLMs. Key mechanism: **self-attention** — allows the model to weigh the importance of each token relative to all others in the input.

```text
Input: "The cat sat on the mat"
                ↓
        Self-Attention
        (which words relate to which?)
                ↓
Output probability: next token
```

Capabilities

- **Text generation** — writing, summarizing, translating
- **Code generation** — writing and explaining code
- **Reasoning** — solving problems step by step
- **Classification** — categorizing text

Limitations

- **Hallucinations** — confident but incorrect information
- **Knowledge cutoff** — no access to real-time data
- **No true understanding** — pattern matching, not comprehension
- **Context limits** — can't process infinite text

Fine-Tuning

Training an already-trained model on specific data to specialize it.

Type	Description
Full fine-tuning	Updates all model weights. Expensive.
LoRA	Updates only a small subset of weights. Efficient.
RLHF	Human feedback to align model behavior with preferences.

RAG (Retrieval-Augmented Generation)

Combines LLM with external data retrieval. The model retrieves relevant documents before generating a response.

```text
User question → Retrieve relevant docs → Feed docs + question to LLM → Ar
```

Why RAG?

- Reduces hallucinations (grounded in real data)
- Updates knowledge without retraining
- Keeps sensitive data private

Temperature

Controls randomness in output.

Value	Behavior
0	Deterministic, most probable token
0.1-0.4	Low creativity, focused
0.7-1.0	Balanced
1.0+	High creativity, more random

Quick Reference

Concept	What it is
Token	Smallest unit of text the model processes
Context Window	Max tokens the model can handle
Prompt	Input text given to the model
Completion	Output generated by the model
Fine-tuning	Specializing a pre-trained model
RAG	Augmenting LLM with external data retrieval
Hallucination	Confident but false output

Prompt Engineering

What is Prompt Engineering?

The practice of designing inputs (prompts) to get desired outputs from LLMs.

Anatomy of a Good Prompt

```text
[Role]        → Who the AI should act as
[Context]     → Background information
[Task]        → What you want it to do
[Format]      → How you want the output
[Constraints] → Rules or limitations
```

Example:

```text
You are a senior React developer.
I have a component that re-renders on every keystroke.
Write a memoized version using useCallback and React.memo.
Use TypeScript. Keep it under 30 lines.
```

Prompting Techniques

Zero-Shot

No examples. Just the task.

```text
Classify this text as positive, negative, or neutral:
"The product is okay but the shipping was slow."
```

Few-Shot

Provide examples before the task.

```text
Classify the sentiment:

Text: "I love this!" → Positive
Text: "Terrible experience" → Negative
Text: "It's fine I guess" → Neutral

Text: "The product is okay but the shipping was slow."
→
```

Chain-of-Thought (CoT)

Ask the model to think step by step.

```text
Solve this step by step:
If a shirt costs $25 and is 20% off, then has an additional 10%
discount at checkout, what is the final price?
```

Role Prompting

Assign a specific role to guide tone and expertise.

```text
Act as a senior backend engineer reviewing this code for security
vulnerabilities. Be specific about each issue found.
```

Structured Output

Explicitly define the format.

```text
List 3 React hooks and their use cases.
Format:
- Hook name: description
```

Best Practices

Be Specific

TEXT

```
❌ "Write a function"
✅ "Write a TypeScript function that takes an array of users and
    returns only active users sorted by name"
```

Provide Context

TEXT

```
❌ "How do I fix this error?"
✅ "I'm using React 18 with TypeScript. When I call setState inside
    useEffect without a dependency array, I get: Warning: useEffect
    cleanup function must return..."
```

Break Down Complex Tasks

TEXT

```
❌ "Build me a full authentication system"
✅ Step 1: "Design the database schema for user auth with email/password"
Step 2: "Write the registration endpoint with validation"
Step 3: "Write the login endpoint with JWT"
```

Use Delimiters

Separate instructions from content.

TEXT

```
Summarize the following article in 3 bullet points:

---
[article text here]
---
```

Common Patterns

For Code Generation

```text
Write a React custom hook called useLocalStorage that:
- Accepts a key and initial value
- Persists to localStorage
- Handles JSON serialization
- Includes TypeScript generics
- Include usage example
```

For Code Review

```text
Review this code for:
1. Security issues
2. Performance problems
3. Best practices violations

Explain each issue with the line number and suggested fix.
```

For Explanations

```text
Explain [concept] like I'm a developer who knows [related concept]
but has never used [target concept]. Use a practical example.
```

Anti-Patterns

Anti-Pattern	Why it's bad
Too vague	Model guesses what you want
Too long	Model may lose focus
Contradictory instructions	Unpredictable output
No examples for complex tasks	Model may misunderstand format
Assuming knowledge	Model doesn't know your codebase

Quick Reference

Technique	When to use
Zero-shot	Simple, clear tasks
Few-shot	Specific format or style needed
Chain-of-thought	Math, logic, complex reasoning
Role prompting	Need expertise in specific domain
Structured output	JSON, tables, specific formats

AI Tools for Developers

Code Assistants

GitHub Copilot

AI pair programmer that suggests code completions and entire functions.

Feature	Description
Inline suggestions	Real-time code completion
Chat	Ask questions about your code
Code review	PR summaries and suggestions
CLI	Terminal command generation

Best for: Daily coding, boilerplate, learning new APIs.

Pricing: Free tier available, Pro $10/mo, Business $19/mo.

Cursor

AI-native code editor built on VS Code with deep AI integration.

Feature	Description
Tab completions	Multi-line code suggestions
Cmd+K	Edit code with natural language
Chat	Context-aware codebase Q&A
Composer	Multi-file changes from instructions

Best for: Large refactors, codebase understanding, complex edits.

Pricing: Free tier, Pro $20/mo.

Windsurf (Codeium)

AI code editor with "Flows" for multi-step tasks.

Best for: Agentic workflows, full-stack features.

Chat Assistants

ChatGPT

General-purpose AI assistant with code capabilities.

Feature	Description
Code generation	Write and explain code
Analysis	Debug and review code
Memory	Remembers context across conversations
Web browsing	Access to current information

Best for: Quick questions, learning, brainstorming.

Pricing: Free tier, Plus $20/mo.

Claude

Anthropic's AI assistant known for longer context and nuanced responses.

Feature	Description
Long context	200K token window
Code analysis	Strong at reviewing and explaining
Artifacts	Generate and preview code/files
Projects	Persistent context for ongoing work

Best for: Long code reviews, documentation, complex analysis.

Pricing: Free tier, Pro $20/mo.

Gemini

Google's AI assistant with multimodal capabilities.

Best for: Integration with Google ecosystem, multimodal tasks.

Specialized Tools

Tool	Purpose
v0.dev	Generate UI components from descriptions
Bolt.new	Full-stack apps from prompts
Replit AI	Code generation in cloud IDE
Tabnine	On-premise code completion

Comparison

Feature	Copilot	Cursor	ChatGPT	Claude
Inline suggestions	✅	✅	❌	❌
Codebase awareness	Partial	✅	❌	Partial
Multi-file edits	❌	✅	❌	❌
Chat interface	✅	✅	✅	✅
Web access	❌	❌	✅	✅
Free tier	✅	✅	✅	✅

Best Practices

Use the Right Tool for the Task

Task	Recommended Tool
Quick code completion	Copilot
Large refactor	Cursor
Debugging help	ChatGPT / Claude
Code review	Claude
Learning a concept	ChatGPT / Claude
UI generation	v0.dev

General Tips

1. **Provide context** — share relevant code, not just the question

2. **Verify outputs** — always review AI-generated code
3. **Iterate** — refine prompts based on results
4. **Use as starting point** — AI generates drafts, you polish
5. **Learn from suggestions** — understand why the AI wrote what it wrote

Quick Reference

Tool	Type	Best For
GitHub Copilot	Code assistant	Daily coding
Cursor	Code editor	Complex refactors
ChatGPT	Chat assistant	Quick Q&A
Claude	Chat assistant	Long context, review
v0.dev	UI generator	Frontend components
Bolt.new	Full-stack	Rapid prototyping